\NAME: Bhavya PS

REG NO: 192524017

COURSE CODE: CSA1716

COURSE NAME: Artificial Intelligence

Artificial Intelligence – Assignment 3

Title: *Implementation of the 8-Puzzle Problem Using A Search Algorithm in Python**

1. Problem Description

The 8-Puzzle Problem consists of eight numbered tiles and one blank space arranged on a 3×3 board. The objective is to move the tiles by sliding them into the blank space until the board matches the goal configuration.

The *A Search Algorithm** is used because it efficiently finds the shortest path from the initial state to the goal state by combining the actual path cost and heuristic estimate.

2. Program Implementation (Python)

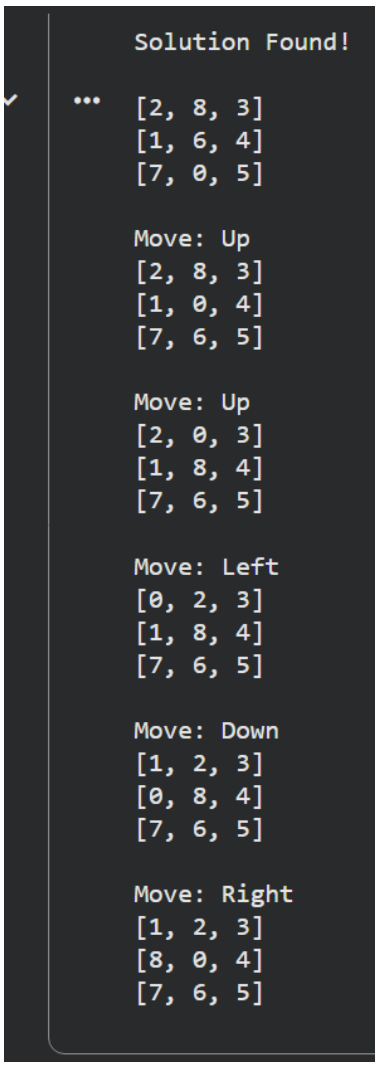
```
import heapq


GOAL = [
    [1, 2, 3],
    [8, 0, 4],
    [7, 6, 5]
]


class Node:
    def __init__(self, state, parent=None, move="", cost=0):
        self.state = state
        self.parent = parent
        self.move = move
        self.cost = cost
        self.heuristic = self.calculate_heuristic()
        self.total = self.cost + self.heuristic


    def calculate_heuristic(self):
        count = 0
```

```
        for i in range(3):
            for j in range(3):
                if self.state[i][j] != 0 and self.state[i][j] != GOAL[i][j]:
                    count += 1
        return count

    def __lt__(self, other):
        return self.total < other.total

def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j

def copy_state(state):
    return [row[:] for row in state]

def get_neighbors(node):
    neighbors = []
    x, y = find_blank(node.state)

    moves = [
        (-1, 0, "Up"),
        (1, 0, "Down"),
        (0, -1, "Left"),
        (0, 1, "Right")
    ]

    for dx, dy, move in moves:
        nx, ny = x + dx, y + dy

        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = copy_state(node.state)
            new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]

            neighbors.append(Node(new_state, node, move, node.cost + 1))
```

```
    return neighbors

def print_state(state):
    for row in state:
        print(row)
    print()

def astar(start):
    open_list = []
    visited = set()

    heapq.heappush(open_list, Node(start))

    while open_list:
        current = heapq.heappop(open_list)

        state_tuple = tuple(map(tuple, current.state))

        if state_tuple in visited:
            continue

        visited.add(state_tuple)

        if current.state == GOAL:
            path = []

            while current:
                path.append(current)
                current = current.parent

            path.reverse()

            print("Solution Found!\n")

            for step in path:
                if step.move:
                    print("Move:", step.move)
                print_state(step.state)
```

```
            return

        for neighbor in get_neighbors(current):
            heapq.heappush(open_list, neighbor)

    print("No Solution Exists")

initial = [
    [2,8,3],
    [1,6,4],
    [7,0,5]
]

astar(initial)
```

3. Input and Output Screenshots

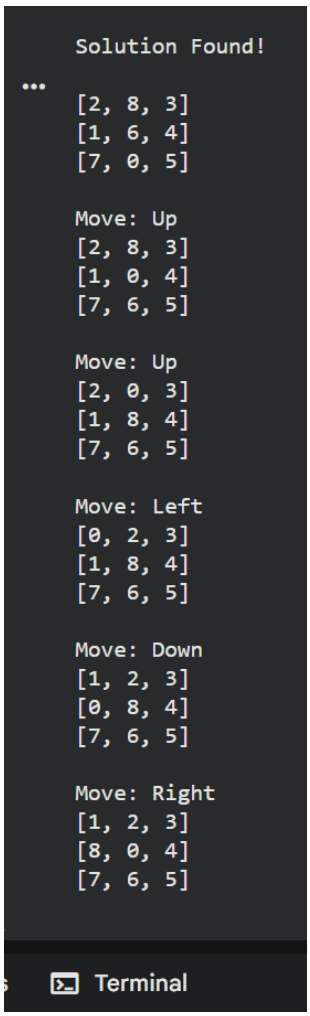


4. Explanation of the Code

- The **Node** class stores the puzzle state, parent node, move, cost, and heuristic value.
- The **heuristic function** counts the number of misplaced tiles.
- **find_blank()** locates the empty tile.
- **get_neighbors()** generates all possible moves (Up, Down, Left, Right).
- *A Search** maintains a priority queue and always expands the state with the lowest **f(n) = g(n) + h(n)**.
- The search continues until the goal state is reached or no solution exists.

5. Test Cases

Goal state reached



Already in Goal State

```
Enter the Initial State (use 0 for the blank tile)
Enter Row 1: 1 2 3
Enter Row 2: 8 0 4
Enter Row 3: 7 6 5

Solution Found!

[1, 2, 3]
[8, 0, 4]
[7, 6, 5]
```

Goal state reached in 1 move

```
Enter the Initial State (use 0 for the blank tile)
Enter Row 1: 1 2 3
Enter Row 2: 8 4 0
Enter Row 3: 7 6 5

Solution Found!

[1, 2, 3]
[8, 4, 0]
[7, 6, 5]

Move: Left
[1, 2, 3]
[8, 0, 4]
[7, 6, 5]
```

6. Applications in Real Life

- Robot Navigation
- GPS Route Planning
- Warehouse Automation
- Video Game AI
- Pathfinding in Autonomous Vehicles
- Puzzle-Solving Software

7. Challenges Faced

- Representing the puzzle state efficiently.
- Implementing the A* priority queue correctly.
- Avoiding repeated exploration of the same states.
- Selecting an effective heuristic function.
- Managing memory usage for large search spaces.

8. Conclusion

The 8-Puzzle Problem demonstrates how Artificial Intelligence uses search algorithms to solve complex problems. The A* Search Algorithm combines the actual path cost with a heuristic estimate to efficiently find the optimal solution. This assignment illustrates the practical application of state-space search and heuristic techniques, which are widely used in robotics, navigation systems, and intelligent software.

9. References

1. Stuart Russell & Peter Norvig, **Artificial Intelligence: A Modern Approach**, 4th Edition, Pearson.
2. Elaine Rich & Kevin Knight, **Artificial Intelligence**, McGraw-Hill.
3. Nils J. Nilsson, **Artificial Intelligence: A New Synthesis**, Morgan Kaufmann.